

Demonstrating `mmap()`, `munmap()` & GPU driver functionality through Pong

Taiyan Liu, Yongqi Shi, Andre Lim, Xinyu Zhang

COMPSCI 510: Fall 2025

The goal of our project was to create a simple Pong game, which would adequately make use of a working implementation of `mmap()`, `munmap()`, as well as GPU driver functionality. This paper will introduce the first two sections in order, and we will then present the mechanism of the Pong game. The GPU driver functionality is unfortunately not fully functional as we are still facing some code issues and it is still work-in-progress.

`mmap()`

The designated fingerprint of `mmap()` is: `void *mmap(void *addr, int len, int prot, int flags, int fd, uint64 offset)`. `mmap()` creates a new mapping in the virtual address space of the calling process. It maps a file into a process's virtual address space.

Normally, we do syscall-backed `read()` and `write()` by commands after calling `int fd = open("dummy.txt", O_RDWR)`, which is a syscall that traps to kernel via `ecall`, finds the metadata-containing inode struct corresponding to `dummy.txt` (in this example), creates a file struct, then in our current calling `proc` struct, assigns a file descriptor number and stores our new struct file in the table denoted by the field `struct file *ofile[NOFILE]`. `mmap()` bypasses the convention detailed above.

In implementing a fundamentally working version of `mmap()`, we largely follow the logical hints presented in the MIT lab assignment. Firstly, each process needs to have an array of virtual memory regions in its virtual address space, where each element describes one mapping from a file to a virtual address range. In our code, this is the field `struct mapped_region mapped_regions[16]` (16 is just a randomly chosen, sufficiently large number).

Each `mapped_region` struct defined in `proc.c` takes the following arguments:

- `uint64 start_address`
- `uint64 end_address`
 - So the mapping covers `[start_address, end_address)`
- `int prot`
 - `PROT_READ` means that the page will be mapped with read permission (`PTE_R`)
 - `PROT_WRITE` means that the page will be mapped with write permission (`PTE_W`)
- `int flags`
 - `MAP_SHARED`: changes in memory must be pushed back to the file on `munmap()`
 - `MAP_PRIVATE`: changes are not written back to the file on `munmap()`

- *uint64 offset*
 - Supposed to be the file offset of the first byte of this mapping, but we ignore it in this lab (i.e. generally take it to be zero)
- *struct file *mapped_file*
 - Pointer to the open file that is mapped by the virtual address range
- *int in_use*
 - 0: this slot is free, 1: this mapping is active

The constants mentioned for the *prot* and *flags* arguments are defined in *fcntl.h* as they usually are. In addition, each *proc* has the field *uint64 mapped_region_top* which is the designated top of our *mmap* region space in the user address space and it is initialised to the extreme *TRAPFRAME* (below *TRAMPOLINE*) to avoid memory conflicts. *mmap()* allocated region starts directly below the trapframe page in virtual address space and each new *mmap()* moves *mapped_region_top* downward.

Our reasoning then leads us to answer the question that if multiple processes map the same file, and they fault on the same file page, they should all use the same physical pages. To do so, we track sharing and freeing with a reference count integer. In *trap.c*, we now define

```
struct mmap_phys_page {
    struct file *f;           // which file this page came from
    uint64 file_page;        // which page index in that file (file_offset / PGSIZE)
    char *mem;               // kernel virtual address of the physical page, the actual physical page containing that
                             // file page
    int refcnt;              // how many mappings point to this page
    int in_use;              // whether this entry is valid
};
```

and the global variable *static struct mmap_phys_page mmap_pages[MMAP_MAX_PAGES]* is declared. This is initialised in *mmap_phys_init()*, and called from *procinit()* at boot. So during kernel boot, before any user processes run, we have already set up the global *mmap_pages[]* variable. All the shared page logic and on-demand mapping lives in *trap.c*. The general idea is that in this array, the key is (*f*, *file_page*) i.e. which file and which page index in that file and the value is physical page *mem* and *refcnt* and *in_use*. We define the helper function *mmap_find_locked(f, file_page)* (also in *trap.c*), which basically just scans through global *mmap_pages* and returns an entry whose *f* and *file_page* match. Note that to do so, we must take the global lock *mmap_pages_lock* because *mmap_pages[]* is shared across CPUs, so there are race conditions.

As mentioned, we need it such that all common mappings of the same *file_page* share one physical page and that it is loaded only one time. Given a file *f* and a byte offset *file_off*, we obtain a physical page that contains that file data, and we make sure that all other processes that request for the same (*f*, *file_off*) share the same physical page, forming a global page cache for *mmap()* file pages. This is essential in our Pong game later. As such, we define *mmap_get_shared_page(struct file *f, uint64 file_off)*, which looks up or creates one of the *mmap_phys_page* in *mmap_pages[]* entries, and returns its *mem* if necessary. The pair (*f*, *file_page*) is what we are using as our key in *mmap_pages[]*. This makes use of the helper

function *mmap_find_locked(f, file_page)*. If an entry is found in the array, If it finds such an entry, it basically means we already created and filled a physical page for that *(file, page)* earlier. So we increment the *refcnt* of the found *struct mmap_phys_page* because there is now another mapping that will use the same physical page. We can then obtain *mem = e->mem*, release the lock, and return it as a pointer.

In the event that such a shared array entry is not found, we have to go through *mmap_pages[]* to find an entry where *in_use = 0* (i.e. a free slot), setting it to 1. We then record the entry's *f* and *file_page*, zero out its *mem*, and set its *refcnt* to be 0. Note that after doing this, we have to release the lock because we do not want to enter a kernel stall because we need to do *kalloc()* of a new page of physical memory (since we do not have a shared array entry). But we still need to take the lock before that to update the metadata and make a reservation of the particular entry so that other threads will observe this. This is a subtle point that we almost forgot so our code is detailed below:

```
char *mmap_get_shared_page(struct file *f, uint64 file_off) {
    if (file_off % PGSIZE != 0) {
        return 0;
    }
    uint64 file_page = file_off / PGSIZE;

    .
    acquire(&mmap_pages_lock);
    struct mmap_phys_page *e = mmap_find_locked(f, file_page);
    if (e) {
        e->refcnt++;
        char *mem = e->mem;
        release(&mmap_pages_lock);
        return mem;
    }

    struct mmap_phys_page *slot = 0;
    for (int i = 0; i < MMAP_MAX_PAGES; i++) {
        if (!mmap_pages[i].in_use) {
            slot = &mmap_pages[i];
            slot->in_use = 1;
            slot->f = f;
            slot->file_page = file_page;
            slot->mem = 0;
            slot->refcnt = 0;
            break;
        }
    }
    release(&mmap_pages_lock);

    if (slot == 0) {
        return 0;
    }
}
```

```

}

char *mem = (char *)kalloc();
if (mem == 0) {
    acquire(&mmap_pages_lock);
    slot->in_use = 0;
    slot->f = 0;
    slot->file_page = 0;
    release(&mmap_pages_lock);
    return 0;
}
memset(mem, 0, PGSIZE);

```

We take the global spinlock *mmap_pages_lock* to safely look at the global *mmap_pages[]* array, then we use our helper function to look through the array. Note that we only execute *kalloc()* if *e* is not null i.e. a shared page already exists for that particular (*file*, *file_page*) key. The fundamental idea is that *kalloc()* has to be executed outside of the range in which we are holding the lock. Note that if *kalloc()* fails (no free memory), we must take the lock again and undo the reservation i.e. set *in_use* to 0 etc. When reading bytes into *mem* from the file and some input or output error occurs, we also have to do *kfree()* of *mem* outside the range in which we hold the lock, then take the lock again and update *in_use* to be 0.

Upon a process exit, there are some things we have to clean up. Note that we modify *freeproc()* in *proc.c* such that for each process exit, for each active *mapped_region* struct, we compute its size *sz* = *end_address* - *start_address*, then we call *mapped_region_unmap()*, a helper function we define in *vm.c* to go through one by one every single page in the range *start_address* to *end_address*, then write back dirty MAP_SHARED pages to the actual file before dropping each shared page reference count and clearing the PTEs. *fileclose(r->mapped_file)* drops the underlying file reference count, before *in_use* is reset to 0 again.

In considering a forked child process, the child process should have identical *mapped_regions*, with the same virtual address ranges, prot, flags, offsets. For each active mapping, we call *filedup(np->mapped_regions[i].mapped_file)* (as hinted from the MIT assignment) in *fork()* for the *mapped_file* so parent & child share the underlying file but refcount is correct. Note that parent and child processes share the same underlying file struct so *filedup()* only needs to be called once for the child process. *mapped_region_top* is the same in parent and child. Actual user pages are copied by the *uvmcopy()* function, then metadata and file references are accounted for as we describe here.

As detailed in the MIT assignment, we need to add code to cause a page-fault in a *mmap()* region to allocate a page of physical memory, read 4096 bytes of the relevant file into that page, and map it into the user address space. This is the thing about the lazy page allocation which we handle and detail here. As such, before we move into the details, we present our *sys_mmap()* function:

```

uint64 sys_mmap(void) {
    struct proc *p = myproc();

```

```

if (p->o_sz == -1) {
    p->o_sz = p->sz;
}

uint64 addr;
int len, prot, flags, fd;
struct file *f;
uint64 offset;

if (argaddr(0, &addr) < 0 || argint(1, &len) < 0 || argint(2, &prot) < 0 || argint(3, &flags) < 0 ||
argfd(4, &fd, &f) < 0 || argaddr(5, &offset) < 0) {
    return 0xffffffff;
}

uint64 all_len = PGROUNDUP(len);
p->mapped_region_top -= all_len;
uint64 start = p->mapped_region_top;
uint64 end = start + all_len;

struct mapped_region *r = &p->mapped_regions[slot];
r->start_address = start;
r->end_address = end;
r->prot = prot;
r->flags = flags;
r->offset = 0;
r->mapped_file = filedup(f);
r->in_use = 1;

return start;
}

```

Here we do not call *mappages()* here, or *kalloc()* for user pages or *readi()* (hinted at in the MIT assignment) to fill any memory with file data. Recall that *kalloc()* allocates one physical page per mapped page. *readi()* synchronously reads every page's data from disk into those pages, and *mappages()* creates valid PTEs in our page table for every page of the region. After *mmap()* returns to user, the end result that we have is that the process now has a virtual address range *[start, end)* recorded in its own *mapped_regions[]* field. However, in the page table *pagetable_t*, those addresses are still unmapped i.e. *walk(pagetable, va, 0)* would return 0 for these virtual addresses. No PTEs exist for these pages yet and this is our model for lazy page allocating. For *sys_mmap()* only, we reserve a virtual address interval *[start, end)* for the mapping, store the mapping info in *p->mapped_regions[slot]* then we immediately return the start address to user space. The result is that after *mmap()*, the process has a pointer *start* that logically refers to that file mapping but the page is not actually mapped yet in that sense.

The function *read_mapping()* is the main *mmap()* page fault handler. When a user touches an *mmap()* address that has not been really mapped yet, hardware raises a page fault. In *usertrap* we have:

```
if ((r_scause() & 0xff) == 13 || (r_scause() & 0xff) == 15) {  
    if (read_mapping(p->pagetable, r_stval()) < 0) {  
        p->killed = 1;  
    }  
}
```

where 13 is load fault, 15 is store fault, then we call our helper *read_mapping()*, which checks if the virtual address input is supposed to be *mmap()*, by going through *mapped_regions[]* and simply checking some bounds. If it is supposed to be *mmap()*, we calculate which file page corresponds to that virtual address, call *mmap_get_shared_page()* to give it a physical page with the file contents, then we call *mmap_pages()* to make the PTE. If mapping by *mmap_pages()* fails, we call *mmap_release_shared_page(mem)* to decrease the *refcount* and free or keep the page.

munmap()

When we call *munmap(addr, len)*, in *sys_munmap()*, we go and find the mapped_region containing *addr* (this is just some basic bound checking). We call the helper function *mapped_region_unmap()* to actually unmap full pages, after aligning the unmap region to page boundaries with *PGROUNDUP*.

mapped_region_unmap() is defined in *vm.c*, it basically walks from *a = addr* to *addr + nbytes* in steps of *PGSIZE*, then we find the PTE with *walk*. If *walk* returns 0, there is no mapping at that page, if PTE has only the *PTE_V* flag, this is a non-leaf entry and if PTE is valid leaf entry (i.e. **pte* and *PTE_V*), we get the physical address *phyaddr*. If the PTE is dirty and the mapping is *MAP_SHARED*, we write changes back to the file using *ilock()*, *iunlock()* and *writei()*. Then we execute the helper function *mmap_release_shared_page()*, which decrements global *refcount* and frees the physical page if this was the last reference (from *trap.c*). The PTE is finally set to 0, fully unmapping that virtual address for this process.

Whenever the user writes to a writeable user page, the CPU always sets *PTE_D* of the PTE in the pagetable to be 1. This is very important. Inside *mapped_region_unmap()*, we have

```
if ((*pte & PTE_D) && (r->flags & MAP_SHARED)) {  
    begin_op();  
    ilock(r->mapped_file->ip);  
    uint64 aoff = a - r->start_address;  
  
    if (aoff < 0) {  
        writei(r->mapped_file->ip, 0,  
            phyaddr + (-aoff),
```

```

        r->offset,
        PGSIZE + aoff);
} else if (aoff + PGSIZE > (r->end_address - r->start_address)) {
    writei(r->mapped_file->ip, 0,
        phyaddr,
        r->offset + aoff,
        r->end_address - r->start_address - aoff);
} else {
    writei(r->mapped_file->ip, 0,
        phyaddr,
        r->offset + aoff,
        PGSIZE);
}

iunlock(r->mapped_file->ip);
end_op();
}

```

Writebacks are basically done with our three functions *ilock()*, *iunlock()*, and *writei()*, which we will do not go into detail for. Basically, if the page is dirty and the mapping is MAP_SHARED, then only will writeback occur. Writeback will occur on *munmap()*.

Testing

We start off by implementing a series of small tests, which we detail in order below.

- test_shared_comm

Creates *shared.txt* containing "00000" and *mmap()* one page with `PROT_READ | PROT_WRITE` and `MAP_SHARED`. After *fork()* to create the child process, it reads 0 at *p[0]*, before it overwrites *p[0]* and *p[1]* with 1, calls *munmap()*, then *exit()*. We have the parent process wait for the child process then we check that the parent does read 1 at *p[0]*. We then call *munmap()*, close and reopen the file to check that our writeback to the file is working. Checks that `MAP_SHARED` allows physical memory (i.e. an actual physical page) to be shared by parent and child processes.

- test_private_cow

Creates *private.txt* containing "AAAAA" and *mmap()* one page with `PROT_READ | PROT_WRITE` and `MAP_PRIVATE`. After *fork()* to create the child process, the child process writes B at *p[0]*, then reads *p[0]* to make sure it is B and then writing is allowed. After *munmap()* and *exit()* are called, the parent process reads *p[0]* and this should be reading as A, not B, thereby confirming that the child's write is on its own private page. Next, the parent process calls *munmap()*, close and reopen the file to check that the file on disk reads A, verifying that `MAP_PRIVATE` writes do not go into writeback.

- test_ro_loophole

Creates *ro_test.txt* with "ZZZZZ", then reopens it as `O_RDONLY` and calls *mmap()* with `PROT_READ | PROT_WRITE` and `MAP_PRIVATE`. The child process writes *p[0]* = X, calls *munmap()*, and closes the file, then reopens and reads *p[0]*, which should be X to pass the test. We allow the private writeable mapping since we call `MAP_PRIVATE` and `PROT_WRITE` but it should not write changes back to disk for a read-only file descriptor.

- test_illegal_map

Checks that we must reject illegal combination of flags: `MAP_SHARED | PROT_WRITE` on a read-only file descriptor. It creates *illegal.txt*, writes "TEST", closes it, and reopens it as `O_RDONLY`. Then we do attempt *mmap(..., PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0)*. Because this would imply writing back to a file that was opened read-only, the kernel should fail the call and return error.

- test_stress_tree

Creates *stress.txt*, then we write S to it, then we *mmap()* one page with `PROT_READ` and `MAP_PRIVATE`, then we call *fork()* to get first child process, which we get it to read *p[0]* = S and then we call *fork()* again to get a grandchild process in some sense. The grandchild also reads *p[0]* = S, then calls *munmap()* and exits. Meanwhile, the original parent also forks a second child that checks *p[0]* = S and unmaps. Finally, the parent waits for both children and unmaps its own mapping. Throughout this entire process, no one writes to the mapping, so all processes should see the same unchanged read-only data.

- test_ro_write_crash

Creates *ro_crash.txt* and writes DATA to it. Now we close the file and reopens it read-only then call *mmap()* to *mmap()* a page as `PROT_READ` only. After *fork()* to create child process, the child process

writes $p[0] = X$, but since this is a read-only mapping; this should trigger a page fault and kill the child process. The parent then calls `wait(&status)` and checks that the status is non-zero; if the child exits with status 0 (meaning it reached the `exit()` after the write), the test fails, since that would mean the kernel allowed a write to a read-only mapping.

- `test_independent_shared`

Creates `indep.txt` and writes to it 11111 and closes the file. Then we call `fork()`. In the parent process, it opens the file for read-write, `mmap()` one page with `MAP_SHARED | PROT_READ | PROT_WRITE`, writes $p[0] = 2$, and then waits for the child process. The child sleeps briefly, then opens `indep.txt` on its own, `mmap()` the same region with `MAP_SHARED` and read $p[0]$ to make sure that it is 2. If the child process still sees 1, that means the kernel did not share a single physical page between independent mappings.

- `test_middle_unmap`

Creates `middle.txt` containing three pages of A, maps the whole 3-page region with `MAP_SHARED`, and then calls `munmap(p + PGSIZE, PGSIZE)` to unmap only the middle page. After that we write $p[0] = X$ (page 1) and $p[2*PGSIZE] = Z$ (page 3), which must succeed to show those pages are still valid. Then we call `fork()` to get a child process that writes $p[PGSIZE] = Y$ into the unmapped middle page, which should cause a crash. The parent process waits and checks that the child is killed with non-zero status. Finally, we call `munmap()` on the remaining two pages.

- `test_ref_count_race`

Creates `ref.txt` and writes to it OLD, calls `mmap()` with `MAP_SHARED | PROT_READ | PROT_WRITE`, and then we call `fork()`. The parent process immediately closes the file descriptor and calls `munmap()` on its mapping, then waits for the child process and then reopens the file. The child sleeps for a bit then we write NEW into the mapping via $p[0:3]$, we call `munmap()`, and then `exit()`. If everything is correct, the mapping's reference keeps the file alive long enough for the child process's writes to reach disk, and the parent process's final read should yield NEW. If the reference counting is wrong, the file might get destroyed or not updated properly, causing the test to fail.

- `test_exec_cleanup`

Creates `exec_test.txt` and writes to it ORIG then calls `fork()` to create a child process. The child process opens the file, calls `mmap()` with `MAP_SHARED`s, `PROT_READ | PROT_WRITE`, closes the file descriptor, then writes DONE into the mapping via $p[0:4]$. After modifying the memory, it calls `exec("echo", argv)` to replace its address space with the echo program. The call to `exec()` should implicitly unmap all regions and flush dirty shared pages to the file. After the child exits, the parent reopens `exec_test.txt` and reads the first 4 bytes, expecting DONE. If the file still contains ORIG, it means the kernel did not propagate shared modified pages on `exec()` teardown.

- `test_same_file_two_shared`

Creates *two_shared.txt* with one page filled with X, *mmap()* that page twice via *p1* and *p2* with `MAP_SHARED | PROT_READ | PROT_WRITE`, and checks that both initially read X. We then write *p1[0] = A* and immediately read to check that *p2[0] = A*, proving they share physical memory. Next we write *p2[1] = B* and read to check that *p1[1] = B*. After unmapping both mappings and closing the file, it reopens *two_shared.txt*, reads back the page, and we expect to read that *page[0] = A* and *page[1] = B*. This test validates that multiple shared VMAs of the same (*fd, offset*) combination converge on a single cached page and that dirty data is written back properly when all are unmapped or closed. This test ensures that two `MAP_SHARED` mappings of the same file region in a single process share physical pages and stay consistent with each other and their writebacks persist to the file.

Pong

For our project demo, we decided to make a simple *Pong* game. This is detailed in *user/game.c*. For the purposes of this report, I will explain it in moderate detail.

The shared state is defined in *game.h* and is designed to be able to fit entirely inside one 4KB page.

```
struct GameState {
    int running;    // 1 while game is running, 0 to quit
    int ball_x;
    int ball_y;
    int ball_vx;
    int ball_vy;
    int paddle1_y; // center y of left paddle
    int paddle2_y; // center y of right paddle
    int p1_up;
    int p1_down;
    int p2_up;
    int p2_down;
};
```

It encodes whether the game is still running (*running*), ball position and velocity (*ball_x*, *ball_y*, *ball_vx*, *ball_vy*), paddle centre positions (*paddle1_y*, *paddle2_y*), one-shot input flags (*p1_up*, *p1_down*, *p2_up*, *p2_down*) that the input process sets and is required by the physics process. This entire struct lives in one page of virtual memory that is shared between processes using *mmap()* (with `MAP_SHARED` of course). We present the function *open_shared_state()*:

```
static struct GameState* open_shared_state(void) {
    shm_fd = open(SHM_FILE, O_CREATE | O_RDWR);
    if (shm_fd < 0) {
        printf("game: open %s failed\n", SHM_FILE);
        return 0;
    }
}
```

```

}

char zero = 0;
for (int i = 0; i < PGSIZE; i++) {
    if (write(shm_fd, &zero, 1) != 1) {
        break;
    }
}

shm_len = PGSIZE;
struct GameState *st = (struct GameState *)mmap(0, shm_len, PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
if (st == (void *)-1 || st == 0) {
    printf("game: mmap failed\n");
    close(shm_fd);
    shm_fd = -1;
    return 0;
}
return st;
}

```

SHM_FILE is “*game.shm*”. We get a file descriptor *shm_fd* that will be used as the backing store for the shared memory. Then we have *struct GameState *st = (struct GameState *)mmap(0, shm_len, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0)*; which returns a pointer to the start of this mapped region, so that now we can be writing into the memory that is backed by the file *game.shm*, and shared among processes. On the other hand, *close_shared_state()* is our cleanup function which cleans up this process’s shared memory mapping via *munmap()* and closes the backing file descriptor. Each process calls *close_shared_state()* when it is done using the shared state.

In our *main()* function in *game.c*, the parent process calls *open_shared_state()*, then it gets a pointer *st* to the shared *GameState* inside the *mmap()* page. The parent process initialises the shared struct by just setting initial values for all fields. Because the mapping is MAP_SHARED, these values are in the shared page. Next, our parent process calls *fork()* twice. After *fork()*, the child processes inherit the same virtual address space, including the *mmap()* region, the same *st* pointer value and the same *shm_fd* file descriptor. Now that in both child processes, *st* points to the same virtual address as in the parent process, that address maps to the same underlying physical page because it is a shared file mapping.

Each process runs a different loop, but all of them operate on the same one single *GameState*:

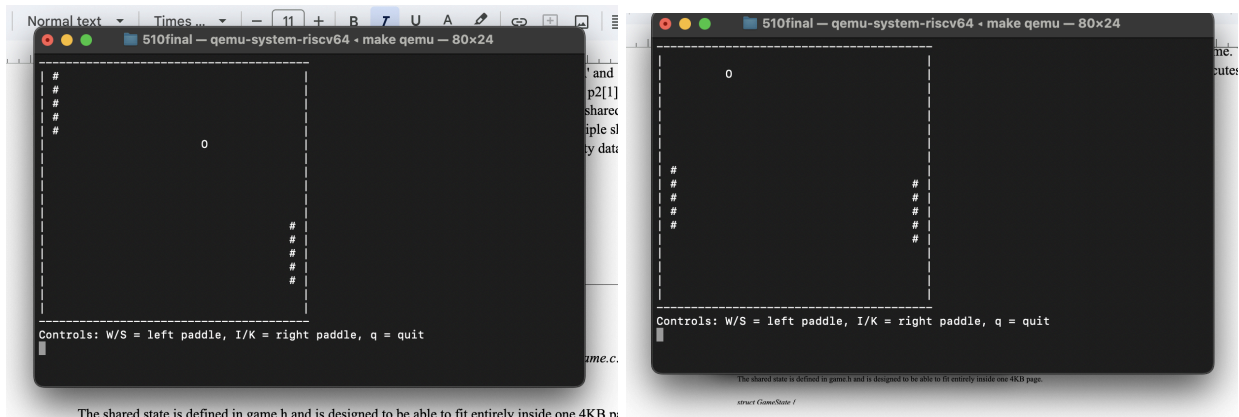
- First child process: *render_loop(st)*
 - Renders the game by reading *st* fields
 - This is rendered to the terminal console
- Second child process: *input_loop(st)*
 - Reads input and sets *st->pl_up*, etc.

- Parent process: *physics_loop(st)*
 - Moves ball and paddles, reads and resets input flags

Any write by one process is immediately visible to the others. For the input process, when the player enters keys like W, S, I, K, this process sets flags in the shared *GameState*. It does not move paddles itself but it instead just marks how paddles should be moving by setting flags to 1. These writes to *st->pl_up* etc. are visible to the physics process. The physics process moves paddles based on input flags, resets those flags (one-shot) and then updates ball position and handles collisions. Finally the render process repeatedly calls *draw_screen(st)*, which clears the terminal using VT100 escape codes, then draws borders, paddles, and ball based on *st->ball_x*, *st->ball_y*, *st->paddle1_y*, etc. in shared memory.

Each process eventually cleans up. The render child process leaves *render_loop(st)* when *st->running* becomes 0 (input process sets running to 0 on 'q' entry). Then it calls *close_shared_state(st)*, which calls *munmap(st, shm_len)* and *close(shm_fd)*, then *exit(0)*. The input child process does the same. The parent physics process stops running *physics_loop(st)* when *st->running* becomes 0. Then it executes *wait(0)* and *wait(0)* to reap the two children, before executing *close_shared_state(st)* and *exit(0)*.

Below are some pictures of the simple Pong game.



Further Work (GPU Functionality)

Our original intention was to develop a fully-fledged Pong game, which would fully demonstrate our *mmap()* functionality, as well as additional GPU capabilities. The goal of our GPU driver was to give xv6 a screen that we can draw on like a real framebuffer, using QEMU's built-in virtio-gpu device i.e. we wanted to write code to talk to the fake GPU that QEMU provides internally, sending frames to QEMU, for QEMU to show in a graphics window. xv6 talks to that device using the virtio ring (just a circular array of descriptor indices) and MMIO registers, just like the existing disk driver. To do this, we built three major pieces:

1. A framebuffer (an array of pixels inside xv6), we #define a particular resolution for it
2. A GPU driver to talk to the virtio-gpu device
3. A drawing API (`fb_clear`, `fb_fillrect`, `fb_flush`) that lets Pong draw its simple graphics

We reuse the virtio infrastructure from *virtio.h* and the existing disk driver:

- *struct virtq_desc*, *struct virtq_avail*, *struct virtq_used* define the layout of the descriptor table and the two rings that the driver and device share. They make up a shared data structure in RAM physical memory that both the driver (xv6) and the device (virtio hardware in QEMU) read and write

The following functions are implemented inside our *virtio_gpu.c*:

1. *virtio_gpu_init*: sets up the GPU device
2. *fb_clear*: clears entire framebuffer to a certain colour
3. *fb_fillrect*: draws a filled rectangle at (x, y) with width w and height h
4. *fb_flush*: sends the framebuffer to the GPU to display it

virtio_gpu.c contains our main GPU driver implementation. We define `VIRTIO_GPU` as the starting address of the GPU's control area in memory, in *memlayout.h*. At offsets from that base GPU MMIO (just a particular memory range that is not RAM) address (0x00, 0x04, 0x08, ...), there are consecutive 32-bit registers. We define the function *RG(r)* to return a pointer to the 32-bit register at GPU base address + offset r.

A virtqueue is just:

1. An array of descriptors (each describes one buffer in memory)
2. An “available” ring that the driver uses to say “here are descriptors I want you (device) to process”
3. A “used” ring that the device uses to say “I’m done with these descriptors; here are the results”

We define *struct gpu_dev* to describe all the state the driver uses to talk to the virtqueue:

- desc – pointer to descriptor table (array of descriptors)
- avail – pointer to “available” ring (what driver publishes)
- used – pointer to “used” ring (what device fills in)
- free[`NUM`] – array that tells us which descriptors are free to use
- lock – spinlock

static struct gpu_dev gpu is the one global instance of *gpu_dev*. We also define *fbmem[]* as the raw pixel data buffer for the screen, just a big array of 32-bit values inside kernel memory, and we define *fb_initialized = 0*, a flag to warn that the GPU is not fully set up yet.

When we boot, we call *virtio_gpu_init()*, which initialises the spinlock inside our global *gpu* with the name *virtio_gpu*. Every time we manipulate *gpu.desc*, *gpu.avail*, etc., we take this lock. We allocate three

pages of memory, one to store the descriptor table (*gpu.desc*), one to store the avail ring (*gpu.avail*), and one to store the used ring (*gpu.used*),

```
gpu.desc = (struct virtq_desc *)kalloc();
gpu.avail = (struct virtq_avail *)kalloc();
gpu.used = (struct virtq_used *)kalloc();
```

where in *xv6*'s existing *virtio.h* (previously already defined), each *struct virtq_desc* tells the device:

- Where a buffer is in physical memory
- How big it is
- What direction data flows (read / write)
- Whether there's another descriptor chained after this one

Those pages are all set to 0 so all descriptors and ring counters start from a clean state. We have fixed identifiers from the *virtio* specification that every *virtio* device must present in its MMIO registers, so the OS can detect it. This is done by sending bus transactions to the GPU device, and we simply read the first 4 registers of the MMIO region and see if they contain the correct values. Note that there are numerous constants here which can be found in the *virtio.h* and the relevant *virtio* specification:

```
if ((*RG(VIRTIO_MMIO_MAGIC_VALUE) != 0x74726976) ||
    (*RG(VIRTIO_MMIO_VERSION) != 2) ||
    (*RG(VIRTIO_MMIO_DEVICE_ID) != VIRTIO_GPU_DEVICE_ID) ||
    (*RG(VIRTIO_MMIO_VENDOR_ID) != 0x554d4551)) {
    printf("virtio_gpu: no device\n");
    return;
}
```

Next, we have to go through the *virtio* handshake:

```
*RG(VIRTIO_MMIO_STATUS) = status;
status |= VIRTIO_CONFIG_S_ACKNOWLEDGE;
*RG(VIRTIO_MMIO_STATUS) = status;
status |= VIRTIO_CONFIG_S_DRIVER;
*RG(VIRTIO_MMIO_STATUS) = status;
```

Next, we configure queue 0, which carries all GPU commands, and we tell the device to use NUM descriptors in the queue:

```
*RG(VIRTIO_MMIO_QUEUE_SEL) = 0;
*RG(VIRTIO_MMIO_QUEUE_NUM) = NUM;
```

This is followed by code which ultimately passes the physical addresses of the three *kalloc()* queue structures we mentioned above, before we mark the queue as ready. The code attaches *fbmem[]* as the

backing and maps that resource to scanout 0, then clearing framebuffer to black and flushing it once. We are now ready to send commands through queue 0.

Frame Rendering

fbclear_color() simply does a double loop over the entire framebuffer. For each row y column x , we compute $index = y * FB_WIDTH + x$, then write colour into *fbmem[index]*. Basically, we just update RAM in the kernel and nothing goes to the GPU.

fb_fillrect(x, y, w, h, color) adjusts the rectangle so it does not go outside the screen. If starting x or y is negative, we shift inwards and shrink width/height. If the right edge or bottom edge goes past the screen, we shrink width/height. If after that width or height is ≤ 0 , there is nothing to draw. Then we double loop through this rectangle and set each pixel's color in *fbmem*.

fb_flush() sends *fbmem* to the GPU. The *struct virtio_gpu_transfer_to_host_2d xfer* command transfers our local pixel buffer to the GPU VRAM. In our code, we construct the little command packet (edit the fields of *xfer*). Then *gpu_cmd(&xfer, sizeof(xfer), &resp, sizeof(resp))* sends the command to the GPU via the virtqueue.

The *struct virtio_gpu_ctrl_hdr resp* command is the response header we get back from the GPU after issuing a command. This is an important function since we have previously only drawn into RAM but the display is not yet updated by the GPU.

Outside of our initialisation function, the game/rendering loop is fundamentally:

```
while (true) {
    handle_input();           // W/S, I/K, etc. to update paddle positions
    update_game_state();      // move ball, detect collisions
    clear_framebuffer();      // memset(fbmem, 0, ...)
    draw_paddles_and_ball();  // write pixels into fbmem at the right positions
    gpu_flush();
}
```

We attempted to write the *gpu_cmd* function but a working version has not been completed yet. Due to this, we render the simplified Pong game with all the same game logic in the terminal console as a temporary measure.

The functional flow for *gpu_cmd* every time is:

1. Lock the GPU state, ensures that the operations on the GPU are serialised and that no other processes can access or modify GPU state during the execution of the command
2. Decide whether we need 1 descriptor (no response) or 2 (command + response)
3. Use *alloc_chain* to grab *ndesc* descriptors from *gpu.free[]*

4. Fill the first descriptor with:
 - *addr = out*
 - *len = out_len*
 - flags possibly with NEXT if there is another.
5. If expecting a response:
 - Fill second descriptor with:
 - *addr = in*
 - *len = in_len*
 - *flags = WRITE*
 - If *in_len* is big enough, cast in to *hdr* and set *hdr->type = 0*.
6. Publish descriptor chain:
 - *gpu.avail->ring[gpu.avail->idx % NUM] = idx[0];*
 - Memory barrier
 - *gpu.avail->idx++*.
7. Notify device:
 - **RG(VIRTIO_MMIO_QUEUE_NOTIFY) = 0;*
8. If this is GET_DISPLAY_INFO:
 - Spin until *hdr->type != 0* or until we time out
9. Else:
 - Do a short empty loop as a tiny delay
10. Free descriptors via *free_chain*
11. Unlock and return